

Day 1 – Programming Fundamentals (50 Questions)

Industrial Assignment 1 | C++ Solutions

> **Submission Format (as required):**

> Each question includes: Problem Statement → Algorithm/Steps → Dry Run → Program Code → Expected Output → Logic Explanation

■ Section 1: Variables (Q1–Q10)

Question 1 — What is a variable in C/C++?

Problem Statement:

Define what a variable is in the context of C/C++ programming.

Answer:

A variable is a named memory location that stores a value which can change during program execution. It has three key properties: a **name** (identifier), a **data type** (what kind of value it holds), and a **value** (the actual data stored).

```cpp

int age = 20; // 'age' is a variable of type int storing value 20

float marks = 95.5; // 'marks' is a variable of type float

```

Logic: The compiler reserves memory space based on the data type, and the variable name is used to access that memory location.

Question 2 — Why do we use variables in programs?

Problem Statement:

Explain the purpose of variables in programming.

Answer:

Variables are used because:

1. They allow programs to store and manipulate data dynamically.
2. They make programs reusable — the same code can work with different inputs.
3. They make code readable by giving meaningful names to values.

```cpp

// Without variable (hardcoded, not reusable):

cout << 5 \* 100;

// With variable (flexible):

```
int price = 5, quantity = 100;
cout << price * quantity;
...
```

**Logic:** Variables act as containers. Instead of hardcoding values, we store them in variables so the program can handle any input.

---

### Question 3 — What is the difference between a variable name and its value?

**Problem Statement:**

Distinguish between a variable's name (identifier) and its value.

**Answer:**

| Aspect     | Variable Name                                             | Variable Value                          |
|------------|-----------------------------------------------------------|-----------------------------------------|
| Definition | The label/identifier used to refer to the memory location | The actual data stored at that location |
| Example    | `age`                                                     | `20`                                    |
| Changes?   | No (fixed after declaration)                              | Yes (can change during execution)       |

```
```cpp
```

```
int age = 20;
// 'age' is the variable name → always refers to this memory location
// 20 is the value → can be changed: age = 25;
...
```

Logic: The name is like a label on a box; the value is the contents inside the box.

Question 4 — Which of the following is a valid variable name? a) 2num b) total_marks c) for d) my marks

Problem Statement:

Identify the valid variable name from the given options.

Answer: b) `total\_marks` is the only valid variable name.

Option	Valid?	Reason
`2num`	■ No	Cannot start with a digit
`total_marks`	■ Yes	Starts with a letter, uses underscore
`for`	■ No	Reserved keyword in C++
`my marks`	■ No	Contains a space

```
```cpp
```

```
int total_marks = 85; // Valid
// int 2num = 5; // Error: starts with digit
// int for = 5; // Error: 'for' is a keyword
// int my marks = 5; // Error: space not allowed
```

```
```
```

****Logic:**** Variable names must start with a letter or underscore, contain only letters/digits/underscores, and must not be a C++ keyword.

Question 5 — Why is `Age` different from `age`?

****Problem Statement:****

Explain case sensitivity in C++ variable names.

****Answer:****

C++ is ****case-sensitive****, meaning uppercase and lowercase letters are treated as distinct characters. `Age` and `age` are two completely different variables in C++.

```
```cpp
```

```
int age = 20;
int Age = 30;
// These are two separate variables stored at different memory locations
cout << age; // prints 20
cout << Age; // prints 30
```

```
```
```

****Logic:**** The compiler distinguishes between ASCII values of uppercase (65–90) and lowercase (97–122) letters, so `A` ≠ `a` internally.

Question 6 — Can a variable name contain spaces? Why?

****Problem Statement:****

Determine whether spaces are allowed in variable names and explain why.

****Answer:****

****No****, a variable name cannot contain spaces.

****Reason:**** The compiler uses spaces as delimiters (separators) to tokenize the code. If a variable name had a space, the compiler would interpret it as two separate tokens, causing a syntax error.

```
```cpp
```

```
int student marks = 90; // ■ Error — compiler reads 'student' and 'marks' as two separate things
int student_marks = 90; // ■ Correct — underscore used instead
```

```
int studentMarks = 90; // ■ Correct — camelCase used

```

**\*\*Logic:\*\*** Use underscore `\_` or camelCase to write multi-word variable names.

---

### Question 7 — Can a variable name start with an underscore (`\_`)?

**\*\*Problem Statement:\*\***

Check if an underscore is a valid starting character for a variable name.

**\*\*Answer:\*\***

**\*\*Yes\*\***, a variable name can start with an underscore (`\_`) and it is syntactically valid in C++.

```
```cpp
```

```
int _count = 10; // ■ Valid
```

```
int _totalMarks = 95; // ■ Valid
```

```
```
```

**\*\*However\*\***, it is generally **\*\*not recommended\*\*** for regular programmers because:

- Names starting with `\_` or `\_\_` (double underscore) are conventionally reserved for compiler and standard library implementations.
- Using them may cause conflicts with system identifiers.

**\*\*Best Practice:\*\*** Start your variable names with a lowercase letter (e.g., `count`, `totalMarks`).

---

### Question 8 — What happens if two variables have the same name in the same scope?

**\*\*Problem Statement:\*\***

Describe the consequence of declaring two variables with identical names in the same scope.

**\*\*Answer:\*\***

It causes a **\*\*compilation error\*\***: "redeclaration of variable". In C++, each variable must have a unique name within the same scope.

```
```cpp
```

```
int main() {
```

```
int marks = 80;
```

```
int marks = 90; // ■ Compile Error: 'marks' was already declared
```

```
return 0;
```

```
}
```

```
```
```

**\*\*However\*\***, in different scopes, the same name is allowed (inner scope shadows outer):

```
```cpp
```

```

int marks = 80; // global scope
int main() {
int marks = 90; // local scope — different variable, shadows the global one
cout << marks; // prints 90 (local)
return 0;
}
...

```

****Logic:**** The compiler tracks all variable names in each scope. Duplicate names within the same scope are ambiguous, so they are not allowed.

Question 9 — Write three meaningful variable names for storing student information.

****Problem Statement:****

Suggest appropriate variable names for student-related data.

****Answer:****

```

```cpp
// Three meaningful variable names for student information:
string studentName; // stores the full name of the student
int rollNumber; // stores the roll number / student ID
float percentage; // stores the student's percentage marks

// Example usage:
studentName = "Rahul Sharma";
rollNumber = 101;
percentage = 87.5;

cout << "Name: " << studentName << endl;
cout << "Roll No: " << rollNumber << endl;
cout << "Percentage: " << percentage << "%" << endl;
...

```

**\*\*Logic:\*\*** Good variable names are descriptive, concise, and follow camelCase convention. They should clearly convey the purpose of the data they hold.

---

### Question 10 — Which naming convention is better and why: `studentMarks` or `sm`?

**\*\*Problem Statement:\*\***

Compare meaningful vs. abbreviated variable naming.

**\*\*Answer:\*\***

**\*\*`studentMarks` is better.\*\***

Criterion	`studentMarks`	`sm`
-----------	----------------	------

--	--	--

Readability	■ Clear meaning	■ Ambiguous
-------------	-----------------	-------------

Maintainability	■ Easy to understand later	■ Hard to debug
-----------------	----------------------------	-----------------

Professionalism	■ Industry standard	■ Poor practice
-----------------	---------------------	-----------------

```
```cpp
```

```
// Hard to understand:
```

```
int sm = 85;
```

```
float gi = 9.2;
```

```
// Easy to understand:
```

```
int studentMarks = 85;
```

```
float gradeIndex = 9.2;
```

```
```
```

**\*\*Logic:\*\*** Code is read far more often than it is written. Meaningful names reduce bugs and make collaboration easier. The extra characters cost nothing — clarity is worth it.

---

## ■ Section 2: Data Types (Q11–Q20)

---

### Question 11 — What is a data type?

**\*\*Problem Statement:\*\***

Define data type in C++.

**\*\*Answer:\*\***

A **\*\*data type\*\*** defines the type of data a variable can hold, how much memory it occupies, and what operations can be performed on it.

| Data Type | Size | Example |
|-----------|------|---------|
|-----------|------|---------|

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

|       |         |             |
|-------|---------|-------------|
| `int` | 4 bytes | `25`, `-10` |
|-------|---------|-------------|

|         |         |        |
|---------|---------|--------|
| `float` | 4 bytes | `3.14` |
|---------|---------|--------|

|          |         |              |
|----------|---------|--------------|
| `double` | 8 bytes | `3.14159265` |
|----------|---------|--------------|

|        |        |              |
|--------|--------|--------------|
| `char` | 1 byte | `'A'`, `'z'` |
|--------|--------|--------------|

|        |        |                 |
|--------|--------|-----------------|
| `bool` | 1 byte | `true`, `false` |
|--------|--------|-----------------|

|          |        |           |
|----------|--------|-----------|
| `string` | varies | `"Hello"` |
|----------|--------|-----------|

```
```cpp
```

```
int age = 20;
```

```
float gpa = 8.5;
char grade = 'A';
bool passed = true;
...
```

Logic: Data types tell the compiler how to interpret the bits stored in memory.

Question 12 — Which data type is used to store whole numbers?

Problem Statement:

Identify the correct data type for integer values.

Answer:

The `int` data type is used to store whole numbers (positive, negative, or zero) without any decimal part.

```
...cpp
int age = 21;
int temperature = -5;
int score = 0;
int students = 60;
...
```

Note: Other integer types also exist: `short` (2 bytes), `long` (4–8 bytes), `long long` (8 bytes) — used when the number is very large or memory is limited.

Logic: `int` typically occupies 4 bytes, storing values from approximately –2.1 billion to +2.1 billion.

Question 13 — Which data type is used to store decimal values?

Problem Statement:

Identify the data type for storing numbers with decimal points.

Answer:

Both `float` and `double` store decimal (floating-point) values.

```
...cpp
float price = 99.99f; // 4 bytes, ~6-7 decimal digits of precision
double pi = 3.14159265; // 8 bytes, ~15-16 decimal digits of precision
...
```

- Use `float` when memory is limited and high precision is not needed.
- Use `double` for scientific calculations needing high precision.

Logic: Floating-point numbers are stored using IEEE 754 standard, with separate bits for sign, exponent, and mantissa.

Question 14 — What is the difference between `float` and `double`?

Problem Statement:

Compare the `float` and `double` data types.

Answer:

Feature	<code>float</code>	<code>double</code>
Size	4 bytes	8 bytes
Precision	~6–7 decimal digits	~15–16 decimal digits
Speed	Slightly faster	Slightly slower
Suffix needed	Yes: <code>3.14f</code>	No: <code>3.14</code>
Use case	Games, graphics	Scientific, financial

```
```cpp
```

```
float f = 3.14159265358979f;
double d = 3.14159265358979;
cout << fixed << setprecision(14);
cout << f << endl; // 3.14159274101257 (less accurate)
cout << d << endl; // 3.14159265358979 (more accurate)
```

```
```
```

Logic: `double` has more bits for the mantissa, so it can represent more decimal digits accurately.

Question 15 — Which data type stores a single character?

Problem Statement:

Identify the data type for storing individual characters.

Answer:

The `char` data type stores a single character, enclosed in single quotes.

```
```cpp
```

```
char grade = 'A';
char initial = 'R';
char symbol = '#';
char digit = '5'; // Note: '5' (char) ≠ 5 (int)
```



```
cout << grade; // prints: A
cout << (int)grade; // prints: 65 (ASCII value of 'A')
...
```

**\*\*Logic:\*\*** `char` stores the ASCII integer value of the character internally. It occupies 1 byte (values 0–127 for standard ASCII).

---

### Question 16 — Which data type stores True/False values?

**\*\*Problem Statement:\*\***

Identify the data type used for logical/boolean values.

**\*\*Answer:\*\***

The **\*\*bool\*\*** data type stores boolean values: `true` (1) or `false` (0).

```
```cpp
bool isPassed = true;
bool isAbsent = false;
int age = 18;
bool isAdult = (age >= 18); // evaluates to true

cout << isPassed; // prints: 1
cout << isAbsent; // prints: 0
cout << boolalpha << isPassed; // prints: true
...
```

****Logic:**** Internally, `bool` is stored as an integer — `1` for true and `0` for false. Any non-zero value is treated as `true`.

Question 17 — What data type is suitable for storing a person's name?

****Problem Statement:****

Choose the appropriate data type for storing a full name.

****Answer:****

****`string`**** is the most suitable data type for storing a person's name.

- `char` can only hold a single character.
- `string` holds a sequence of characters (a text/word/sentence).

```
```cpp
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {
string name = "Rahul Sharma";
cout << "Name: " << name << endl; // prints: Rahul Sharma
cout << "Length: " << name.length(); // prints: 12
return 0;
}
...

```

**\*\*Logic:\*\*** `string` in C++ is a class from the `` library that manages a dynamic array of characters, making it flexible for names of any length.

---

### Question 18 — Predict the output: `int age = 18; cout << age;`

**\*\*Problem Statement:\*\***

Predict what the given code snippet will print.

**\*\*Algorithm:\*\***

1. Declare integer variable `age` and assign value `18`.
2. Output the value of `age` using `cout`.

**\*\*Dry Run:\*\***

- `age = 18` → memory stores `18`
- `cout << age` → reads value `18` from memory and prints it

**\*\*Program Code:\*\***

```
```cpp
#include <iostream>
using namespace std;

int main() {
int age = 18;
cout << age;
return 0;
}
...

```

****Expected Output:****

...

18

...

****Logic:**** `cout` reads the value stored in the variable `age` (which is `18`) and displays it on the screen. No newline is added since `endl` is not used.

Question 19 — Identify the data type: `25`, `3.14`, `A`, `true`

Problem Statement:

Match each literal value to its correct data type.

Answer:

Value	Data Type	Reason
----- ----- -----		
`25`	`int`	Whole number, no decimal
`3.14`	`double` (or `float`)	Has decimal point
`A`	`char`	Single character in single quotes
`true`	`bool`	Boolean value

```cpp

int a = 25;

double b = 3.14;

char c = 'A';

bool d = true;

cout << a << " " << b << " " << c << " " << d;

// Output: 25 3.14 A 1

```

Logic: C++ infers the data type from the literal's format: quotes → char, decimal → double, no decimal → int, `true`/`false` → bool.

Question 20 — What happens if you store a decimal number in an `int` variable?

Problem Statement:

Explain the effect of assigning a floating-point value to an integer variable.

Answer:

The **decimal part is truncated (cut off)** — it is NOT rounded, just removed.

Algorithm:

1. Declare `int` variable.
2. Assign a decimal value.
3. The compiler drops everything after the decimal point.

Dry Run:

- `int x = 9.99` → stored as `9` (`.99` is lost)

- `int y = 3.1` → stored as `3`

****Program Code:****

```
```cpp
#include <iostream>
using namespace std;

int main() {
 int x = 9.99;
 int y = 3.1;
 int z = -4.7;

 cout << x << endl; // 9
 cout << y << endl; // 3
 cout << z << endl; // -4
 return 0;
}
```
```

****Expected Output:****

```
```
9
3
-4
```
```

****Logic:**** This is called ****implicit type conversion (narrowing conversion)****. The compiler may show a warning. Always use the correct data type to avoid losing precision in your data.

■ Section 3: Input/Output (Q21–Q30)

Question 21 — What is the purpose of `cin`?

****Problem Statement:****

Explain the role of `cin` in C++.

****Answer:****

`cin` (Character INput) is the ****standard input stream**** in C++. It reads data entered by the user from the keyboard and stores it into a variable.

```
```cpp
#include <iostream>
using namespace std;

int main() {
```

```
int age;
cout << "Enter your age: ";
cin >> age; // reads value from keyboard into 'age'
cout << "You entered: " << age;
return 0;
}
...
```

**Logic:** `cin` is an object of the `istream` class, defined in `<iostream>`. It uses the extraction operator `>>` to take input and assign it to a variable.

---

### Question 22 — What is the purpose of `cout`?

**Problem Statement:**

Explain the role of `cout` in C++.

**Answer:**

`cout` (Character OUTput) is the **standard output stream** in C++. It displays data (text, values of variables, results) on the screen.

```
```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    int marks = 95;
    cout << "Marks: " << marks << endl;
    return 0;
}
...

```

Expected Output:

...

Hello, World!

Marks: 95

...

Logic: `cout` is an object of the `ostream` class, defined in `<iostream>`. It uses the insertion operator `<<` to send data to the screen.

Question 23 — Which symbol is used with `cin`?

****Problem Statement:****

Identify the operator used with ``cin``.

****Answer:****

The ****extraction operator ``>>``** is used with ``cin``.

```
```cpp
int num;
cin >> num; // reads one value

int a, b;
cin >> a >> b; // reads two values (space-separated input)
```
```

****Memory Aid:**** ``>>`` points INTO the variable — data flows from the keyboard into the variable.

****Logic:**** The ``>>`` operator extracts (reads) data from the input stream and places it into the specified variable.

Question 24 — Which symbol is used with ``cout``?

****Problem Statement:****

Identify the operator used with ``cout``.

****Answer:****

The ****insertion operator ``<<``** is used with ``cout``.

```
```cpp
cout << "Hello"; // outputs text
cout << 42; // outputs number
cout << "Age: " << 20; // chained output
```
```

****Memory Aid:**** ``<<`` points AWAY from the variable — data flows from the variable to the screen.

****Logic:**** The ``<<`` operator inserts (sends) data into the output stream to be displayed on screen.

Question 25 — Explain: ``cin >> age;``

****Problem Statement:****

Interpret the statement ``cin >> age;``.

****Answer:****

This statement ****reads an integer value from the keyboard**** and stores it in the variable ``age``.

****Step-by-step breakdown:****

- `cin` → the standard input stream (keyboard)
- `>>` → extraction operator (reads from stream into variable)
- `age` → the variable where the input value will be stored

****Dry Run:****

- Program pauses and waits for user input.
- User types `25` and presses Enter.
- Value `25` is stored in the memory location named `age`.

````cpp`

```
#include <iostream>
using namespace std;

int main() {
 int age;
 cout << "Enter age: ";
 cin >> age;
 cout << "Age is: " << age;
 return 0;
}
```

**\*\*Expected Output (if user enters 25):\*\***

`````

Enter age: 25

Age is: 25

`````

`---`

**### Question 26 — Explain: `cout << age;`**

**\*\*Problem Statement:\*\***

Interpret the statement `cout << age;`.

**\*\*Answer:\*\***

This statement **\*\*reads the value stored in `age`\*\*** and **\*\*displays it on the screen\*\***.

**\*\*Step-by-step breakdown:\*\***

- `cout` → the standard output stream (screen)
- `<<` → insertion operator (sends data to screen)
- `age` → the variable whose value is being printed

**\*\*Dry Run:\*\***

- Suppose `age = 20`.

- `cout << age` → reads value `20` from memory → displays `20` on screen.

```
```cpp
#include <iostream>
using namespace std;

int main() {
    int age = 20;
    cout << age; // prints: 20
    cout << endl;
    cout << "Age: " << age; // prints: Age: 20
    return 0;
}
```
```

**Expected Output:**

```
20
Age: 20
```

---

### Question 27 — What is the purpose of `endl`?

**Problem Statement:**

Explain the use of `endl` in C++.

**Answer:**

`endl` serves **two purposes**:

1. **Inserts a newline character** (`\n`) — moves the cursor to the next line.
2. **Flushes the output buffer** — forces any buffered output to be written to the screen immediately.

```
```cpp
cout << "Line 1" << endl;
cout << "Line 2" << endl;
cout << "Line 3";
```

// vs using `\n` (only newline, no flush):

```
cout << "Line 1\n";
cout << "Line 2\n";
```
```

**Expected Output:**



...

Line 1

Line 2

Line 3

...

**\*\*Logic:\*\*** `endl` = `\n` + `flush`. For most programs, both work the same. `endl` is slightly slower due to flushing but ensures output is displayed immediately — important in real-time applications.

---

### Question 28 — Write a statement to input a student's marks.

**\*\*Problem Statement:\*\***

Write a C++ input statement to read a student's marks.

**\*\*Algorithm:\*\***

1. Declare a variable to hold marks.
2. Prompt the user.
3. Read the input using `cin`.

**\*\*Program Code:\*\***

```cpp

#include <iostream>

using namespace std;

int main() {

int marks;

cout << "Enter student's marks: ";

cin >> marks;

cout << "Marks entered: " << marks << endl;

return 0;

}

```

**\*\*Dry Run (if user enters 87):\*\***

- `marks` gets value `87`

- Output: `Marks entered: 87`

**\*\*Expected Output:\*\***

...

Enter student's marks: 87

Marks entered: 87

...

**\*\*Logic:\*\*** ``cin >> marks`` reads the integer typed by the user and stores it in the ``marks`` variable.

---

### Question 29 — Write a statement to display 'Welcome to C++'.

**\*\*Problem Statement:\*\***

Write a C++ output statement to display the text "Welcome to C++".

**\*\*Program Code:\*\***

```
```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Welcome to C++" << endl;
    return 0;
}
```
```

**\*\*Expected Output:\*\***

---

Welcome to C++

---

**\*\*Logic:\*\*** ``cout << "Welcome to C++"`` uses the insertion operator to send the string literal to the standard output stream (screen). The text is enclosed in double quotes because it is a string.

---

### Question 30 — Predict the output: ``cout << "Programming";``

**\*\*Problem Statement:\*\***

Predict what the statement ``cout << "Programming";`` will output.

**\*\*Algorithm:\*\***

1. ``cout`` sends the string ``"Programming"`` to the screen.
2. No ``endl`` is used, so the cursor stays on the same line.

**\*\*Program Code:\*\***

```
```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Programming";
    return 0;
}
```

```
}  
...
```

****Dry Run:****

- `"Programming"` is a string literal.
- `cout <<` inserts it into the output stream.
- It prints character by character: P-r-o-g-r-a-m-m-i-n-g.

****Expected Output:****

```
...
```

Programming

```
...
```

****Logic:**** `cout` prints the exact content of the string literal. Since there is no `endl` or `\n`, the output appears on a single line with no newline at the end.

```
---
```

■ Section 4: Operators (Q31–Q40)

```
---
```

Question 31 — What is an operator?

****Problem Statement:****

Define an operator in C++.

****Answer:****

An **operator** is a symbol that tells the compiler to perform a specific operation on one or more operands (values or variables).

| Category | Operators | Example |

|-----|-----|-----|

| Arithmetic | `+`, `-`, `*`, `/`, `%` | `a + b` |

| Relational | `==`, `!=`, `<`, `>`, `<=`, `>=` | `a > b` |

| Logical | `&&`, `||`, `!` | `a && b` |

| Assignment | `=`, `+=`, `-=` | `a = 5` |

| Increment/Decrement | `++`, `--` | `a++` |

```
```cpp
```

```
int a = 10, b = 3;
```

```
cout << a + b; // 13 (arithmetic operator)
```

```
cout << (a > b); // 1 (relational operator returns bool)
```

```
...
```

**\*\*Logic:\*\*** Operators are the building blocks of expressions. Without them, programs cannot perform calculations or make decisions.

---

### Question 32 — Which operator is used for addition?

**Problem Statement:**

Identify the addition operator in C++.

**Answer:**

The `+` operator is used for addition.

````cpp`

`#include <iostream>`

`using namespace std;`

`int main() {`

`int a = 15, b = 7;`

`int sum = a + b;`

`cout << "Sum = " << sum << endl; // Sum = 22`

`// Also works with other data types:`

`float x = 3.5, y = 1.5;`

`cout << "Float Sum = " << x + y; // Float Sum = 5`

`return 0;`

`}`

`````

**Expected Output:**

`````

Sum = 22

Float Sum = 5

`````

**Logic:** The `+` operator adds the values of its two operands. It works with `int`, `float`, `double`, and even `string` (for concatenation).

---

### Question 33 — What is the result of `10 % 3`?

**Problem Statement:**

Calculate the result of the modulo operation `10 % 3`.

**Answer:** The result is `1`.

**Algorithm:**

1. Divide 10 by 3:  $10 \div 3 = 3$  remainder `1`.

2. The `%` operator returns the **remainder**.

**\*\*Dry Run:\*\***

-  $10 = 3 \times 3 + 1$

- Quotient = 3, Remainder = **\*\*1\*\***

**\*\*Program Code:\*\***

```
```cpp
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int result = 10 % 3;
```

```
cout << "10 % 3 = " << result << endl;
```

```
return 0;
```

```
}
```

```
```
```

**\*\*Expected Output:\*\***

```
```
```

```
10 % 3 = 1
```

```
```
```

**\*\*Logic:\*\*** The modulo operator `%` gives the remainder of integer division. It is useful for checking even/odd numbers, cyclic patterns, etc.

---

### Question 34 — What is the result of  $8 \% 2$ ?

**\*\*Problem Statement:\*\***

Calculate the result of  $8 \% 2$ .

**\*\*Answer:\*\*** The result is **\*\*0\*\***.

**\*\*Algorithm:\*\***

1. Divide 8 by 2:  $8 \div 2 = 4$  with remainder 0.

2. `%` returns the remainder  $\rightarrow 0$ .

**\*\*Dry Run:\*\***

-  $8 = 2 \times 4 + 0 \rightarrow \text{Remainder} = \mathbf{0}$

**\*\*Program Code:\*\***

```
```cpp
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
cout << "8 % 2 = " << 8 % 2 << endl;
return 0;
}
...
```

****Expected Output:****

...

8 % 2 = 0

...

****Logic:**** When a number is perfectly divisible, the remainder is 0. This is how we check if a number is ****even****: `if (n % 2 == 0)`.

Question 35 — Which operator checks equality?

****Problem Statement:****

Identify the equality-checking operator in C++.

****Answer:****

The `==` (double equals) operator checks if two values are equal. It returns `true` (1) if equal, `false` (0) if not.

```cpp

#include <iostream>

using namespace std;

int main() {

int a = 10, b = 10, c = 5;

cout << (a == b) << endl; // 1 (true — they are equal)

cout << (a == c) << endl; // 0 (false — not equal)

return 0;

}

...

**\*\*Expected Output:\*\***

...

1

0

...

**\*\*Logic:\*\*** `==` is a **\*\*relational operator\*\*** used for comparison. Never confuse it with `=` (assignment operator). `a = 5` stores 5 in `a`; `a == 5` checks if `a` is 5.

---

### Question 36 — What is the difference between `=` and `==`?

**Problem Statement:**

Distinguish between the assignment operator `=` and the equality operator `==`.

**Answer:**

| Feature | `=` (Assignment)              | `==` (Equality)     |
|---------|-------------------------------|---------------------|
| Purpose | Assigns a value to a variable | Compares two values |
| Returns | The assigned value            | `true` or `false`   |
| Usage   | `int x = 5;`   `if (x == 5)`  |                     |
| Type    | Assignment operator           | Relational operator |

```cpp

```
int x = 10; // ASSIGNS 10 to x
if (x == 10) { // CHECKS if x equals 10
    cout << "x is 10" << endl;
}
```

// Common mistake:

```
if (x = 5) { // ■■ This ASSIGNS 5 to x (not comparison!)
    // This will always be true (non-zero)
}
```

Logic: Confusing `=` with `==` is one of the most common bugs in C++. Always use `==` inside conditions.

Question 37 — What is the result of `5 > 3`?

Problem Statement:

Evaluate the relational expression `5 > 3`.

Answer: The result is `true` (printed as `1` in C++).

Algorithm:

1. Compare 5 and 3 using the ``>`` (greater than) operator.
2. Since 5 is indeed greater than 3, the expression evaluates to `true`.

Program Code:

```
```cpp
#include <iostream>
using namespace std;
```

```
int main() {
cout << (5 > 3) << endl; // prints: 1
cout << boolalpha << (5 > 3) << endl; // prints: true
return 0;
}
...
```

**\*\*Expected Output:\*\***

```
...
1
true
...
```

**\*\*Logic:\*\*** Relational operators compare two values and return a boolean result. `true` is displayed as `1` by default, or as `true` with `boolalpha`.

---

### Question 38 — What does `&&` represent?

**\*\*Problem Statement:\*\***

Explain the `&&` operator in C++.

**\*\*Answer:\*\***

`&&` is the **\*\*Logical AND\*\*** operator. It returns `true` only if **\*\*both\*\*** conditions are true.

**\*\*Truth Table:\*\***

A	B	A && B
false	false	false
false	true	false
true	false	false
true	true	true

```cpp

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int marks = 75, attendance = 80;
```

```
if (marks >= 40 && attendance >= 75) {
```

```
cout << "Eligible for exam" << endl;
```

```
} else {
```

```
cout << "Not eligible" << endl;
```



```

}
return 0;
}
...

```

****Expected Output:****
 ...

Eligible for exam
 ...

****Logic:**** `&&` is used when ALL conditions must be satisfied simultaneously. If either condition is false, the whole expression is false.

Question 39 — What does `||` represent?

****Problem Statement:****
 Explain the `||` operator in C++.

****Answer:****
 `||` is the ****Logical OR**** operator. It returns `true` if ****at least one**** condition is true.

****Truth Table:****

| A | B | A B |
|-------|-------|-----------------|
| false | false | false |
| false | true | **true** |
| true | false | **true** |
| true | true | **true** |

```

```cpp
#include <iostream>
using namespace std;

int main() {
 bool hasCash = false;
 bool hasCard = true;

 if (hasCash || hasCard) {
 cout << "Can make payment" << endl;
 } else {
 cout << "Cannot make payment" << endl;
 }

 return 0;
}

```

```
}
...
```

```
Expected Output:
...
```

```
Can make payment
...
```

**\*\*Logic:\*\*** `||` is used when ANY one of the conditions being true is sufficient. It returns false only if ALL conditions are false.

---

### Question 40 — What does `!` represent?

**\*\*Problem Statement:\*\***

Explain the `!` operator in C++.

**\*\*Answer:\*\***

`!` is the **\*\*Logical NOT\*\*** operator. It **\*\*reverses\*\*** (negates) a boolean value — `true` becomes `false`, and `false` becomes `true`.

**\*\*Truth Table:\*\***

A	!A
---	----
true	false
false	true

```
```cpp
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    bool isRaining = false;
```

```
    if (!isRaining) {
```

```
        cout << "Go outside!" << endl;
```

```
    } else {
```

```
        cout << "Stay inside." << endl;
```

```
    }
```

```
    cout << !true << endl; // prints: 0
```

```
    cout << !false << endl; // prints: 1
```

```
    return 0;
```

```
}
```

```
...
```

****Expected Output:****

...

Go outside!

0

1

...

****Logic:**** `!` is a unary operator (works on one operand). It flips the boolean value — useful for checking the opposite of a condition.

■ Section 5: Programs (Q41–Q50)

Question 41 — Write a program to print 'Hello World'.

****Problem Statement:****

Write a C++ program that displays "Hello World" on the screen.

****Algorithm:****

1. Include the `` header.
2. Use `using namespace std`.
3. In `main()`, use `cout` to print "Hello World".
4. Return 0.

****Dry Run:****

- Program starts at `main()`.
- `cout << "Hello World"` sends the string to the output stream.
- `endl` moves cursor to next line.
- `return 0` ends the program successfully.

****Program Code:****

```cpp

#include <iostream>

using namespace std;

int main() {

cout << "Hello World" << endl;

return 0;

}

```

****Expected Output:****

...

Hello World

...

****Logic:**** `cout` is the standard output object. The `<<` operator inserts the string into the output stream, displaying it on the console. This is the most basic C++ program, demonstrating the minimum structure: headers, namespace, main function, and output statement.

Question 42 — Write a program to input two numbers and display their sum.

****Problem Statement:****

Write a C++ program that takes two integers as input from the user and displays their sum.

****Algorithm:****

1. Declare two integer variables `a` and `b`.
2. Prompt the user to enter two numbers.
3. Read both numbers using `cin`.
4. Calculate `sum = a + b`.
5. Display the sum using `cout`.

****Dry Run (Input: a=15, b=27):****

- `a = 15`, `b = 27`

- `sum = 15 + 27 = 42`

- Output: `Sum = 42`

****Program Code:****

```
```cpp
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
 int a, b, sum;
```

```
 cout << "Enter first number: ";
```

```
 cin >> a;
```

```
 cout << "Enter second number: ";
```

```
 cin >> b;
```

```
 sum = a + b;
```

```
 cout << "Sum = " << sum << endl;
```

```
 return 0;
```

```
}
```

```
...
```

**\*\*Expected Output:\*\***

```
...
```

Enter first number: 15

Enter second number: 27

Sum = 42

```
...
```

**\*\*Logic:\*\*** The `+` arithmetic operator adds the two input values. The result is stored in `sum` and displayed using `cout`. Variables `a`, `b`, and `sum` are all declared as `int` since we deal with whole numbers.

```

```

### Question 43 — Write a program to calculate the area of a rectangle.

**\*\*Problem Statement:\*\***

Write a C++ program that takes the length and width of a rectangle as input and calculates and displays the area.

**\*\*Algorithm:\*\***

1. Declare variables `length`, `width`, and `area` (float for decimal support).
2. Prompt user to enter length and width.
3. Read values using `cin`.
4. Calculate `area = length * width`.
5. Display the area.

**\*\*Dry Run (length=7.5, width=4.0):\*\***

- `area = 7.5 * 4.0 = 30.0`

- Output: `Area of Rectangle = 30`

**\*\*Program Code:\*\***

```
```cpp
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    float length, width, area;
```

```
    cout << "Enter length of rectangle: ";
```

```
    cin >> length;
```

```
    cout << "Enter width of rectangle: ";
```

```
    cin >> width;
```

```
    area = length * width;
```

```

cout << "Area of Rectangle = " << area << endl;

return 0;
}
...

```

****Expected Output:****
 ...

```

Enter length of rectangle: 7.5
Enter width of rectangle: 4.0
Area of Rectangle = 30
...

```

****Logic:**** Area of a rectangle = length × width. We use `float` instead of `int` to handle decimal dimensions. The `\*` multiplication operator computes the area.

Question 44 — What formula is used to calculate Simple Interest?

****Problem Statement:****

State the mathematical formula for Simple Interest.

****Answer:****

$$SI = \frac{P \times R \times T}{100}$$

Where:

- ****P**** = Principal amount (initial amount of money)
- ****R**** = Rate of interest per year (in percentage)
- ****T**** = Time period (in years)
- ****SI**** = Simple Interest

****Example:****

- Principal = ₹1000
- Rate = 5% per annum
- Time = 2 years
- SI = $(1000 \times 5 \times 2) / 100 = ₹100$

```cpp

```

float si = (P * R * T) / 100.0;
...

```

**\*\*Logic:\*\*** Simple Interest is directly proportional to principal, rate, and time. It is called "simple" because interest is only calculated on the original principal, not on accumulated interest.

---

### Question 45 — Write a program to calculate Simple Interest.

**Problem Statement:**

Write a C++ program that reads principal, rate, and time and calculates simple interest.

**Algorithm:**

1. Declare variables `P`, `R`, `T` (float) and `SI` (float).
2. Read values of P, R, T from user.
3. Calculate  $SI = (P * R * T) / 100$ .
4. Display the Simple Interest.

**Dry Run (P=5000, R=8, T=3):**

-  $SI = (5000 * 8 * 3) / 100$   
-  $SI = 120000 / 100$   
-  $SI = 1200$   
- Output: Simple Interest = 1200

**Program Code:**

```
```cpp
#include <iostream>
using namespace std;

int main() {
    float P, R, T, SI;

    cout << "Enter Principal amount: ";
    cin >> P;

    cout << "Enter Rate of interest (per year %): ";
    cin >> R;

    cout << "Enter Time period (in years): ";
    cin >> T;

    SI = (P * R * T) / 100.0;

    cout << "Simple Interest = " << SI << endl;

    return 0;
}
```
```

**Expected Output:**

```
Enter Principal amount: 5000
Enter Rate of interest (per year %): 8
```

Enter Time period (in years): 3

Simple Interest = 1200

...

**\*\*Logic:\*\*** The formula  $(P * R * T) / 100$  is applied directly. We use `100.0` (float) in the denominator to ensure floating-point division rather than integer division, preserving decimal accuracy.

---

### Question 46 — Why is a temporary variable used while swapping two numbers?

**\*\*Problem Statement:\*\***

Explain the role of a temporary variable in the swapping algorithm.

**\*\*Answer:\*\***

When swapping two variables `a` and `b`, we need a **temporary variable** because:

- If we directly do `a = b`, the original value of `a` is **permanently lost** before it can be copied to `b`.

**\*\*Without temp (WRONG approach):\*\***

...

`a = 10, b = 20`

`a = b` → `a = 20` (original `a=10` is LOST!)

`b = a` → `b = 20` (still 20, swap failed!)

...

**\*\*With temp (CORRECT approach):\*\***

...

`a = 10, b = 20`

`temp = a` → `temp = 10` (save `a`'s value)

`a = b` → `a = 20` (overwrite `a` safely)

`b = temp` → `b = 10` (restore original `a` to `b`)

Result: `a = 20, b = 10` ■

...

**\*\*Logic:\*\*** The temporary variable acts as a "safe holding area" to prevent data loss during the exchange. It's like using a third cup when exchanging liquids between two cups.

---

### Question 47 — Swap the values: `a=10, b=20`.

**\*\*Problem Statement:\*\***

Write a C++ program to swap two variables `a = 10` and `b = 20`.

**\*\*Algorithm:\*\***



1. Declare `a = 10`, `b = 20`, and `temp`.
2. Store `a` in `temp`.
3. Assign `b` to `a`.
4. Assign `temp` to `b`.
5. Display both values.

**\*\*Dry Run:\*\***

| Step         | `a` | `b` | `temp` |
|--------------|-----|-----|--------|
| Initial      | 10  | 20  | —      |
| `temp` = `a` | 10  | 20  | 10     |
| `a` = `b`    | 20  | 20  | 10     |
| `b` = `temp` | 20  | 10  | 10     |

**\*\*Program Code:\*\***

```
```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 20, temp;

    cout << "Before Swap: a = " << a << ", b = " << b << endl;

    temp = a;
    a = b;
    b = temp;

    cout << "After Swap: a = " << a << ", b = " << b << endl;

    return 0;
}
```
```

**\*\*Expected Output:\*\***

```
```
Before Swap: a = 10, b = 20
After Swap: a = 20, b = 10
```
```

**\*\*Logic:\*\*** Three assignment statements using a temporary variable safely exchange the values of `a` and `b` without losing any data.

---

### Question 48 — What formula is used to convert Celsius to Fahrenheit?

**Problem Statement:**

State the mathematical formula to convert temperature from Celsius to Fahrenheit.

**Answer:**

$$F = \left(C \times \frac{9}{5}\right) + 32$$

Or equivalently:

$$F = (C \times 1.8) + 32$$

Where:

- **C** = Temperature in Celsius

- **F** = Temperature in Fahrenheit

**Example values:**

| Celsius (°C) | Fahrenheit (°F) |
|--------------|-----------------|
|--------------|-----------------|

|   |    |
|---|----|
| 0 | 32 |
|---|----|

|     |     |
|-----|-----|
| 100 | 212 |
|-----|-----|

|    |      |
|----|------|
| 37 | 98.6 |
|----|------|

|     |     |
|-----|-----|
| -40 | -40 |
|-----|-----|

```
```cpp
```

```
float F = (C * 9.0 / 5.0) + 32;
```

```
```
```

**Logic:** The formula accounts for two differences between the scales: the size of one degree (9/5 ratio) and the offset in zero point (32°F = 0°C).

---

### Question 49 — What is the Fahrenheit value of 0°C?

**Problem Statement:**

Calculate the Fahrenheit equivalent of 0 degrees Celsius.

**Answer:** 32°F

**Calculation:**

-  $F = (C \times 9/5) + 32$

-  $F = (0 \times 9/5) + 32$

-  $F = 0 + 32$

-  $F = 32^\circ\text{F}$

**Program Code:**

```

```cpp
#include <iostream>
using namespace std;

int main() {
    float celsius = 0.0;
    float fahrenheit;

    fahrenheit = (celsius * 9.0 / 5.0) + 32;

    cout << celsius << "°C = " << fahrenheit << "°F" << endl;

    return 0;
}
```

```

**\*\*Expected Output:\*\***

```

```
0°C = 32°F
```

```

**\*\*Logic:\*\*** 0°C is the freezing point of water. The Fahrenheit scale places this at 32°F because Fahrenheit set 32 as the freezing point of water in his original scale.

---

### Question 50 — Why can integer division give incorrect results in the Celsius-to-Fahrenheit program?

**\*\*Problem Statement:\*\***

Explain why using integer division in the Celsius-to-Fahrenheit formula causes incorrect results.

**\*\*Answer:\*\***

In C++, when both operands of `/` are integers, the result is also an integer (decimal part is **\*\*truncated\*\***).

**\*\*The problem:\*\***

```

```cpp
// WRONG — integer division:
int celsius = 37;
int fahrenheit = (celsius * 9 / 5) + 32;
// 37 * 9 = 333
// 333 / 5 = 66 (integer! actual = 66.6, .6 is LOST)
// 66 + 32 = 98 ← WRONG (correct answer is 98.6)
```

```

**\*\*The fix — use floating-point values:\*\***

```

```cpp
// CORRECT — floating-point division:
float celsius = 37.0;
float fahrenheit = (celsius * 9.0 / 5.0) + 32;
// 37.0 * 9.0 = 333.0
// 333.0 / 5.0 = 66.6 (decimal preserved!)
// 66.6 + 32 = 98.6 ■ CORRECT
```

```

**\*\*Full Correct Program:\*\***

```

```cpp
#include <iostream>
using namespace std;

int main() {
    float celsius, fahrenheit;

    cout << "Enter temperature in Celsius: ";
    cin >> celsius;

    fahrenheit = (celsius * 9.0 / 5.0) + 32;

    cout << celsius << "°C = " << fahrenheit << "°F" << endl;

    return 0;
}
```

```

**\*\*Expected Output (input: 37):\*\***

```

Enter temperature in Celsius: 37
37°C = 98.6°F

```

**\*\*Logic:\*\*** Always use `float` or `double` variables, and ensure constants like `9.0` and `5.0` are written as floats to force floating-point division. Integer division silently discards the fractional part, causing hard-to-detect precision errors.

---

## ## ■ Summary

| Section | Questions | Topic |

|-----|-----|-----|

| 1 | Q1–Q10 | Variables |

| 2 | Q11–Q20 | Data Types |

| 3 | Q21–Q30 | Input/Output (cin/cout) |

| 4 | Q31–Q40 | Operators |

| 5 | Q41–Q50 | Programs |

---

\*All programs written in C++ | Compiled and tested | Industrial Assignment 1 — Day 1 Programming Fundamentals\*